

A technique report about drawing Sankey diagrams in Java

XuChen 2257453

Introduction:

In today's area of information explosion, we are confronted with massive amounts of data every day. Understanding and analyzing this data are crucial for making informed decisions, and this is where data visualization comes into play. Data visualization involves presenting abstract data in an intuitive and easily understandable form through charts and graphics, enabling us to delve deeper into the patterns and trends within the data.

In the realm of data visualization, three key points merit attention:

*----**Data Cleaning:** Before embarking on data visualization, it's crucial to ensuring the dataset is clean and accurate.*

*----**Interactivity of Charts:** Users should have the ability to select different subsets of data to meet diverse analytical needs. Also, detailed information should be provided which can enhance users' understanding. Allowing users to adjust chart parameters facilitates a better exploration of different aspects of the data.*

*----**Aesthetic Value:** Consideration should be given to the reasonable use of color and labels' size.*

This report focuses on the Sankey diagram visualization. The Sankey diagram, with its flow-line format, clearly illustrates relationships and flows between various elements.

When creating a Sankey diagram, two key points are worth considering:

*----**Data Flow should be affected by numbers and attributes:** Ensure that the Sankey diagram clearly illustrates the flow of data, emphasizing relationships between different components through variations in width and color.*

*----**Clear Labels and Annotations:** Clear labels and annotations should be added to the nodes and flow lines of the Sankey diagram to explain elements in the chart, aiding the audience in understanding key information.*

In this report ,I will discuss 9 chapter to illustrate what I have done associated with these key points:

- 1 Code Design combining with Objective-oriented Principles*
- 2 Algorithm about Data cleaning(File Handling)*
- 3 Diagram Display Algorithm*
- 4 Display while Resizing Algorithm*

5 Algorithm about Additional Functions

6 Exception Handling

7 Code Clarity

8 My Java Code

My contributions:

1 Create a GUI which can meet users' different demands.(Adjust the slope of the curve,the opacity of the curve ,the opacity of the attribute,the word size of the labels/choose different files without close the window and choose different color as every time the diagram is coloured randomly/save the Image made by users)

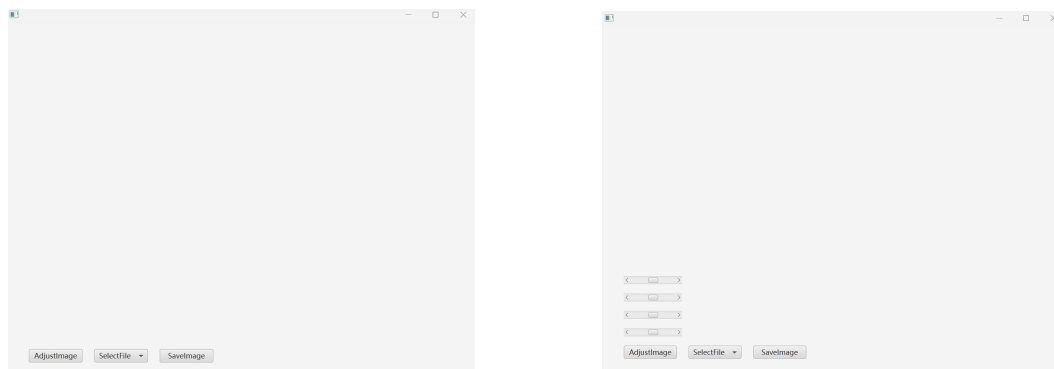


Figure 1 :The window for user interactivity

The following is the four example picture downloaded by SaveImage Button.

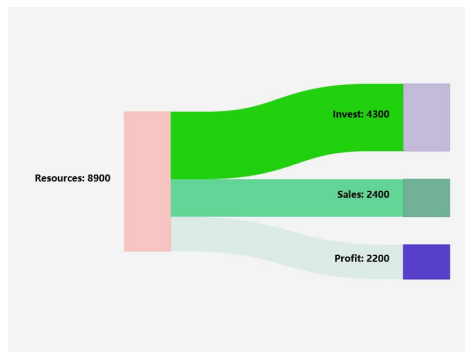


Figure 2 : example1

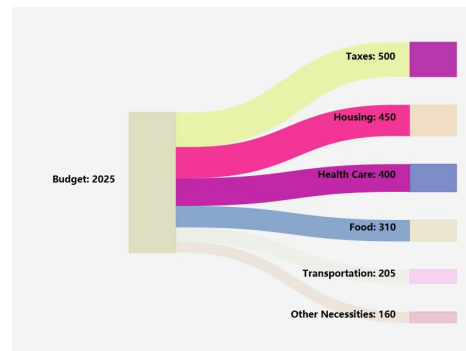


Figure 3 : example2

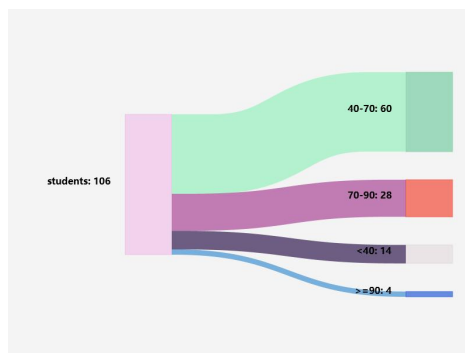


Figure 4 : example3

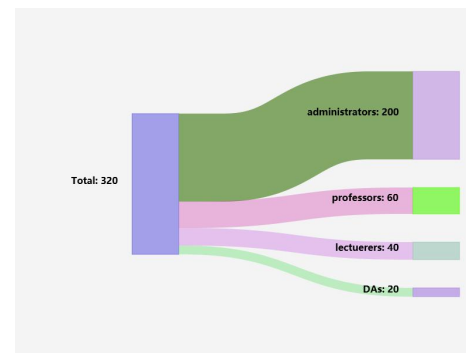


Figure 5 : example4

2 Automatically shrink or magnify the label and diagram when the user resize the window

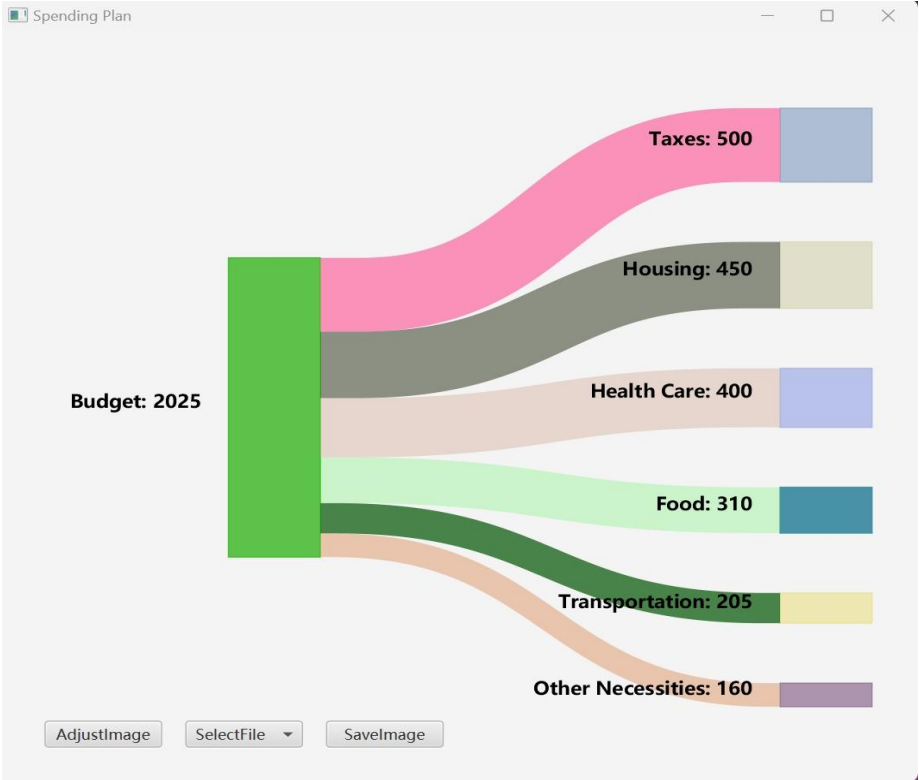


Figure 6 : The original size of example2

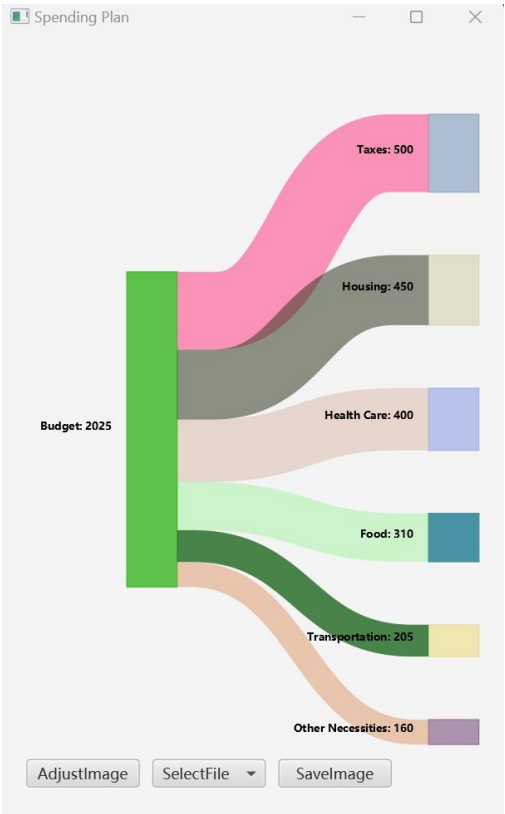


Figure 7 : The diagram when the width shrinks

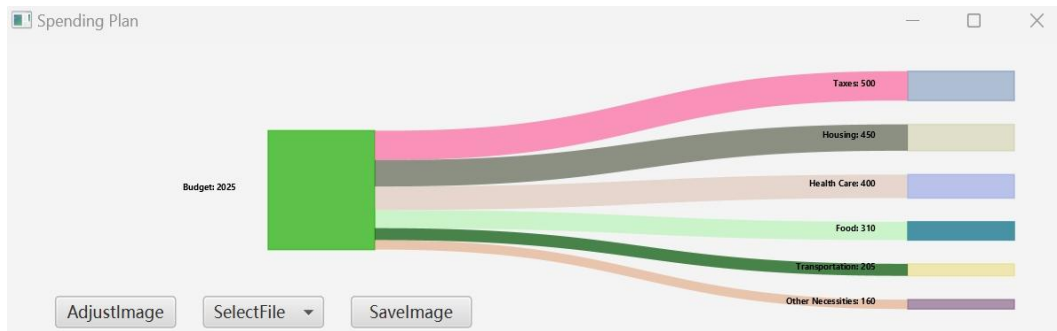


Figure 8 : The diagram when the height shrinks

3 According to the previous diagram, you can see the order of the right ones are in a regular order from biggest to the smallest

1 Code Design combining with Objective-oriented Principles

1.1 Code design

Object-oriented programming (OOP) is a programming paradigm that utilizes fundamental principles to design and organize code. In this chapter, I will explore the effective application of key OOP principles in the context of my Java code for drawing Sankey diagrams.

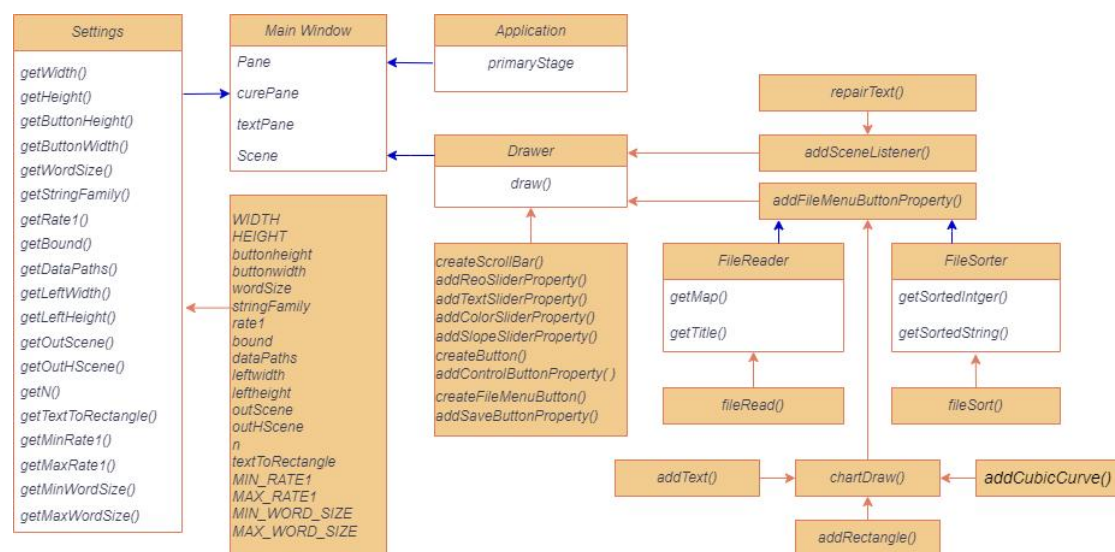


Figure 9 : The general diagram about code. The arguments are not shown in this picture. The blue line means it uses public function or extends from the super class. The orange line means it's a private function.

1.1.1 The purpose of Main Window class(CW3_2257453_sankeydiagrams class)

According to the order, we know that we need to draw a picture with javafx in a window. So there must be a Main Window class, which its purpose is to override the start function extending from Application class, create the basic needs about the window and accept parameters and the function to draw picture.

1.1.2 The purpose of Settings class

As for the parameters, we can create a new list to store all parameters that we need and output with getter function to help its safety, which can not be changed easily by the users.

1.1.3 The reason of creating Drawer class

As for the function to draw a picture, we need to create a new class called Drawer. The reason why we need to create a new class, not to continue writing in the original one is that we need to ensure the Main Window Class has only the one task, which is combine all the things and output. So the draw function is another task, which sets dependency from Main Window class.

1.1.4 The general content of the draw() in Drawer class

According to my imagination, my code have three button: adjust button, file choose button and save image button. The adjust button can make the 4 sliders visible and invisible: curve slope adjust slider, rectangle adjust opacity slider, test size adjust slider, curve opacity adjust slider. The file choose button can show the file based on the list and draw the chart while choosing the label. The save image button can save the picture of the window without showing buttons and sliders.

Upon with the following, we can abstract the previous into **several tasks**:

- 1 create sliders associated with labels and location and set sliders property
- 2 create adjust button and sets property associated with sliders
- 3 create menu button and sets property associated with the file that have been dealt with and the function that can output the file. Also, in order to avoid the previous one will influence the future one, we need have logic with the Scene to clean all the things.
- 4 create save image button and sets property associated with buttons and sliders to set them invisible while shooting.
- 5 What's more, we need to resize the diagram, so there exists a function to monitor the scene.

1.1.5 The details of task4

---How can we get data that have been dealt with? We need to have the class to read the file and sort the file. According to OOP, these tasks are independent, so it need to be separated into two class.

---How can we draw the chart? According to the sankey diagram in task sheet, we can see that there are three components to create a diagram: Text, Rectangle and curve. So we just need three function to create each one.

1.1.5 The details of task5

For monitor the scene, we need this to just adjust the text size because the size of curve and rectangle can be set by initially combine the location with the scene.

1.1.6 The reason why we just need to create a Drawer class in drawing

We just instantiate the several class in javafx:Button,ScrolBar,MenuButton,Text, Rectangle and so on. What we need to do is to ensure the hierarchy of the function to avoid duplicate information. According to OOP principles, combinations are better than inheritance and the function here is sequential and linear, so there is no need to set a super class and override the function again and again.

1.2 The analysis of Code Design in Object-oriented Principles

1.2.1 Encapsulation

Encapsulation helps hide implementation details and promotes loose coupling between classes. The Code Design makes good use of encapsulation principles in the following ways:

---In Settings class, we set all the initial number as private ,which can not be obtained by users and users can only get the value using various getter function that is designed to be static. What's more ,in order to ensure that the picture is appropriate and it can be adjusted by button,we don't need to write setter function.

---In Main Window class, it just run the final code ,so there is no encapsulation in it ,which just call public function.

---In FileReader and FileSorter class, we will get the local path or the map. In FileReader,We set the function that deal with the path and output the map as private function,which can keep the map to be safe.Also,We set the sorting function to be private in FileSorter.What's more,we just set getter function as public and set the storage of inputs as private to keep the value of inputs cannot be easily changed.

---In Drawer class, only the draw function is public and the others handling functions are private just avoid be used by other classes and keep the code safe.

These make the code extensible and code maintainable.

1.2.2 Inheritance and Polymorphism

The only inheritance in this code is the Main Window extends from javafx Application. The aim of this is to override the start function to show the diagram.I have illustrated why we just need to create a Drawer class in drawing (1.1.6), so it does not have inheritance.For FileReader and FileSorter class , they achieve different task, inheriting will destroy the dependency and FileSorter does not need title.

There is no need for Polymorphism in this code,as written before :combinations are better than inheritance in OOP design and the code just has one inheritance.

1.2.3 Abstraction

From figure 9, we can see the code design has done lots of abstraction.Except the Main Window,no class else have access to the true value, which are replaced variable.

Also,in Drawer class, we can see the large code is separated by small steps with hierarchy associated with its task meaning (1.1.4),which make complex concepts into

simpler representations.

The use of abstraction enhances code readability and comprehensibility.

1.2.4 Modularity and Dependency

From figure 9, It's easy to see that the code is designed in modularity that each function and class achieve some meanings(1.1.1-1.1.4).

For dependency, we can see each module is a one-to-many relation and single row with others, which means each of the module have deterministic task and there doesn't exist Circular Dependency problems in this code.

1.2.6 Class Design

Above all ,we can admit that my code class design is reasonable. The property and relationship between each class have been written in 1.1.1-1.1.4. The structure is clear and useful. What's more ,you can add other property into this class without destroy the relationship ,which means the class design is maintainable and extendable.

1.3 A conclusion about several advantages when using OOP in Code

---The code have a clear hierarchy which help programmer to understand and read. Also, developers can abstract the things ,which help understand the logic about each variable without pay more time on numerical computing.

---It sets a certain gap between users and developers, which can ensure the inner variable cannot be changed by users so that developers can keep data safe.

---The code have many modules, which make complex tasks into several easy ones. In other words, developers can override the original function without destroy the structure, which can make project extendable and maintainable.

2 Algorithm about Data cleaning(File Handling)

I have adjust the order about chapter in task sheet in order to illustrate more reasonable. In previous introduction ,I have written that data cleaning is a key point in data visualization. In code ,it's composed of FileReader and FileSorter classes.

2.1 The Algorithm about file read

*The class FileReader accept one argument: **String datapath** and have two private variable : **String title** and **Map<String, Integer> map** ,two public function: **getMap()** and **getTitle()** and one private function: **fileRead()**. The map and title are defined when executing the private function.*

*How does fileRead() function works? Please watch Figure 10. There are somethings need to make up: The pseudo code does not show the error message sending but it does exist in true code. Most importantly, **name.isEmpty()** is to prevent some example that have several lines with nothing. Without this step, many code will not run successfully.*

Algorithm 1 fileRead Function

```
1: function FILEREAD(datapath)
2:   Initialize the map, FullEnitity and sum
3:   input  $\leftarrow$  datapath
4:    $i \leftarrow 0$ 
5:   while input.hasNextLine() do
6:     name  $\leftarrow$  input.nextLine()
7:     if name.isEmpty() then
8:       break
9:     end if
10:    if  $i \geq 2$  then
11:      Split the name into namall and num
12:      Put nameall and num into map
13:       $sum \leftarrow sum + num$ 
14:    end if
15:    if  $i = 1$  then
16:      FullEnitity  $\leftarrow$  name
17:    end if
18:    if  $i = 0$  then
19:      this.title  $\leftarrow$  name
20:    end if
21:     $i \leftarrow i + 1$ 
22:  end while
23:  Put FullEninty and sum into map
24:  return map
25: end function
```

Figure 10: The pseudo code of fileRead function

2.2 The Algorithm about file sort

Algorithm 2 fileSort Procedure

```
1: procedure FILESORT(map)
2:    $len \leftarrow map.values().size()$ 
3:   for  $j \leftarrow 0$  to  $len - 1$  do
4:      $maxnum \leftarrow 0$ 
5:     for i in map.values() do
6:       if  $maxnum < i$  then
7:          $maxnum \leftarrow i$ 
8:       end if
9:     end for
10:    sortedInteger.add(maxnum)
11:    for k in map.keySet() do
12:      if map.get(k) = maxnum then
13:        sortedString.add(k)
14:        map.remove(k)
15:        break
16:      end if
17:    end for
18:  end for
19: end procedure
```

Figure 11: The pseudo code of fileSort function

The class `FileSorter` accept one argument: **`Map<String, Integer> map`** and have two private variable : **`List<Integer> sortedInteger`** and **`List<String> sortedString`** ,two public function: `getSortedInteger()` and `getSortedString()` and one private function: `fileSort()`.The `sortedInteger` and `sortedString` are defined when executing the private function. Additionally,I need to mention that after the `fileSort()` execution,the map is {},which can save the space.

3 Diagram Display Algorithm

According to my code design in Figure 9 , the `chartDraw` function achieves diagram display. I will introduce the details of `addCubicCurve`, `addText`, `addRectangle`, `chartDraw` .

3.1 How to draw curves ?

In sankey diagram,we need to draw curves.In the code ,we choose to draw a Cubic Curve .Cubic Curve is one of the commonly used curve types in computer graphics. It is a cubic Bessel curve defined by four points. These four points include two anchor points (starting and ending points) and two control points. By adjusting the positions of these control points, various curves of different shapes can be created

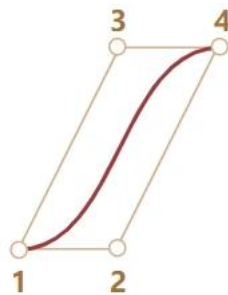


Figure 12: A example of Cubic Curve

In our program we need to make sure the start point 1 and control point 2 have the same height,so as the end point 4 and control point 3 ,which can ensure that the diagram is vertical when we set the stroke width.

The function accept four arguments **`addCubicCurve(Pane pane,Double startY,Double endY,Double startRectangle,Double Rate_leftright)`** and return a **`CubicCurve`** class. We set each of the 4 point location and it will not like the actual one as we set stroke width which is a square,that means the point deviate what we see by eyes. What's more we set the color ,and make sure the `setfill` function is opacity to avoid problems.

3.2 The way to draw Rectangle and Text.

In `addRectangle` function ,it accepts arguments: **`DoubleBinding xProperty`**, **`DoubleBinding yProperty`**,**`DoubleBinding wProperty`**,**`DoubleBinding hProperty`** and

return a **Rectangle** class. We set the location and its width and height. Also, we set the color, but make sure the stroke and filling have the same color to make diagram more clear.

In `addText` function, it accepts arguments: **String textString**, **DoubleBinding xProperty**, **DoubleBinding yProperty** and return a **Text** class. We set the location and make sure that it is bold and italic. Most importantly, we compute the `textWidth` in this function and subtract in `xProperty`, so we just need input the end width location of text.

3.3 The way to combine all the above and draw the chart

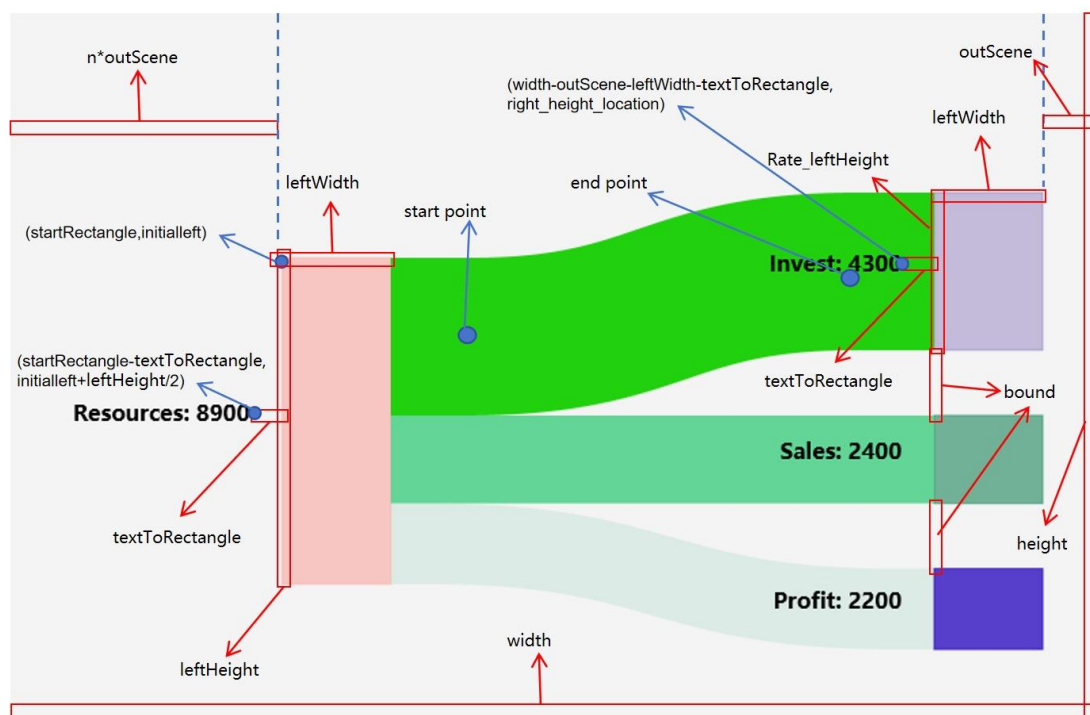


Figure 13: An analysis graphic of example1

Variable setting:

`width`: Scene width

`height`: Scene height

`outScene`: a distance to keep the rectangle away from scene.

`n`: a rate in left to ensure the text is among the scene

`startRectangle`: $n * outScene$

`Initialleft`: the yproperty of left rectangle

`leftWidth`: the width of all rectangles

`leftHeight`: the height of left rectangles

`rate`: the rate of right rectangle values/ left rectangle values

`Rate_leftHeight`: $rate * leftHeight$

`bound`: the bound of right rectangles

CubicCurve location computing:

---Start point:

Initialline: each left-upper height of curve

X: startRectangle+leftWidth+Rate_leftHeight/2

Y:initialline+0.5*Rate_leftHeight

---End point:

right_height_location:each right-upper height of curve

X:width-leftWidth-outScene-Rate_leftHeight/2

Y:right_height_location+0.5*Rate_leftHeight

The algorithm of chartDraw

Algorithm 3 chartDraw Procedure

```
1: procedure CHARTDRAW(primaryStage, pane, textPane, curvePane, data)
2:   initialleft  $\leftarrow 0.5 \times (\text{height} - \text{leftHeight})$ 
3:   startRectangle  $\leftarrow n \times \text{outScene}$ 
4:   r1  $\leftarrow$  call addRectangle()
5:   curvePane.getChildren().add(r1)
6:   text1  $\leftarrow$  call addText()
7:   textPane.getChildren().add(text1)
8:   len  $\leftarrow$  data.getSortedInteger().size()
9:   right_height_location  $\leftarrow 0.5 \times (\text{height} - \text{leftHeight} - (\text{len} - 2) \times \text{bound})$ 
10:  initialline  $\leftarrow$  initialleft
11:  for i  $\leftarrow 1$  to len-1 do
12:    rate  $\leftarrow$  data.getSortedInteger().get(i)/data.getSortedInteger().get(0)
13:    Rate_leftHeight  $\leftarrow$  rate  $\times$  leftHeight
14:    r2  $\leftarrow$  call addRectangle()
15:    curvePane.getChildren().add(r2)
16:    text2  $\leftarrow$  call addText()
17:    textPane.getChildren().add(text2)
18:    startY  $\leftarrow$  initialline +  $0.5 \times \text{Rate\_leftHeight}$ 
19:    endY  $\leftarrow$  right_height_location +  $0.5 \times \text{Rate\_leftHeight}$ 
20:    curve1  $\leftarrow$  call addCubicCurve()
21:    curvePane.getChildren().add(curve1)
22:    initialline  $\leftarrow$  initialline + rate  $\times$  leftHeight
23:    right_height_location  $\leftarrow$  right_height_location + bound +
      Rate_leftHeight
24:  end for
25:  pane.getChildren().addAll(curvePane, textPane)
26:  textPane.toFront()
27: end procedure
```

Figure 14: The pseudo code of chartDraw function

4 Display while Resizing Algorithm

We need two function to achieve this purpose: **chartDraw** and **repairText** (Figure 9). I will explain later about the challenges I have met and how I overcame it.

In **chartDraw**, we use **bind()** combined with **pane.widthProperty()** and **pane.heightProperty()** to reach the resization of location and magnitude of recatangles and curves and the resiztion of location of text. We know all of the variable have been set as "a*width/height" except the text wordsize. So we need to compute and change it into the pattern "a*width/height+b" pattern and compute them use **add()**.

In **repairText**, we first compute the **scaleFactor**, which is a $\min\{\text{newVal}/\text{width}, \text{newVal}/\text{height}\}$ and **newVal** comes from monitor. Then, we use the **(maxWordSize+minWordSize) /2 * scaleFactor** to update the word size. Although we change the word size, the old text location hasn't been change so we need to add this gap just in "b" pattern.

5 Algorithm about Additional Functions

In the introduction paragraph, I have written several key points in data visualization and sankey diagram. We have discuss **Data Cleaning, Data Flow should be affected by numbers and attributes ,Clear Labels and Annotations** in previous chapter. In this chapter, we will mainly focus on **Interactivity of Charts and Aesthetic Value**.

5.1 Adjust Button

In 1.1.4 ,I have written the tasks of it: 1 create sliders associated with labels and location and set sliders property. 2 create adjust button and sets property associated with sliders. Before talking about this, we need to know there are three panes : **pane**, which store the buttons and sliders, **textPane**, which stores the text and **curvePane** ,which stores the rectangles and curves. We use this structure to ensure the button appears in the Scene forever and the text is above the chart.

We first set the location of sliders use **createScrollBar** function, which accepts arguments: **String tooltip, DoubleBinding layoutX, DoubleBinding layoutY, double min, double max, double value**. Then, each of them is designed to add property (**curve slope adjust slider, rectangle adjust opacity slider, test size adjust slider, curve opacity adjust slider**) function to change the variable directly in node of related pane. After that, we use **createButton** to create adjust button and execute **setOnAction** to remove or add sliders into the pane.

5.2 File Menu Button

The **createFileMenuButton** method creates a **MenuButton** object with the location and make sure the page is opened to upper direction. The **addFileMenuButtonProperty** method adds functionality to the fileMenuButton. It creates **MenuItem** objects for each file, assigns an action to each menu item, and adds them to the fileMenuButton's list of items. The action performed when a menu

item is clicked includes removing certain elements from the pane, clearing other panes, reading data from a file, and updating the chart and window title.

5.3 Save Image Button

We can reuse the **createButton** to create the save image button and the **addSaveButtonProperty** method adds functionality to the button. When the button is clicked, it hides certain elements such as the control button, sliders, and file menu button. It then takes a snapshot of the specified pane using the **snapshot** method and opens a file chooser dialog to allow the user to select where to save the image. The method checks that the file extension is valid and saves the image to the selected file using the **write()** method. Finally, the method shows the hidden elements again.

There are something need to say: This need to import `javax.imageio.ImageIO` and it's used only one times in all of my code ,aiming to achieve the save image task .**It's not a subclass of Swing or awt.** In order to achieve the image download ,**you must use this function which can not be replaced by others as javafx have no function about it**, but it provides the class **WritableImage** and **FileChooser** to do the initial steps. You can see the Proof of web link :

<https://stackoverflow.com/questions/74777479/how-to-save-image-in-javafx>

Or javafx tutorial:

<https://www.javatpoint.com/javafx-tutorial>

6 Exception Handling

In `FileReader` class, it will handle the exception and stop execute by **System.exit(0)**

If the format is wrong, such as no value in lines ,it will output "Data format exception at line " + (i + 1)

If the number in lines is negative or 0 ,it will output "Data must be positive"

If the number in lines has two values or more with space (which means it not a title combined with number) ,it will out put "Data overflow"

If the code cannot get the file ,it will output a `IOException`.

7 Code Clarity

I will introduce the Naming method here. All the classes are words combined with upper case in first letter and its solution function is just lower the first letter. To catch the private value, we all use "get+..." naming way. For methods, they are words combined with upper case except the first one. All the name is named according to its actual meaning.

8 My Java Code

8.1 CW3_2257453_sankeydiagrams.java

```
package main;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.stage.Stage;

public class CW3_2257453_sankeydiagrams extends Application {

    @Override

    public void start(Stage primaryStage) {

        Pane pane =new Pane();

        Pane curvePane = new Pane();

        Pane textPane = new Pane();

        Scene scene = new Scene(pane, Settings.getWidth(),Settings.getHeight());

        Drawer drawer= new Drawer(Settings.getWidth(),Settings.getHeight(), Settings.getOutScene(),
Settings.getStringFamily() ,

        Settings.getBound(),Settings.getN() , Settings.getLeftHeight() ,
Settings.getLeftWidth() ,Settings.getWordSize() ,Settings.getRate1() ,

        Settings.getTextToRectangle(), Settings.getButtonHeight(),
Settings.getButtonWidth() ,Settings.getMaxWordSize() ,Settings.getMinWordSize() ,

        Settings.getDataPaths() ,Settings.getOutHScene(),Settings.getMinRate1() ,Settings.getMaxRate1());

        drawer.draw(pane, curvePane, textPane, primaryStage, scene);

        primaryStage.setScene(scene);

        primaryStage.show();

    }

    public static void main(String[] args) {

        launch(args);

    }

}
```

8.2 Drawer.java

```
package main;

import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import javafx.beans.property.DoubleProperty;
import javafx.beans.value.ObservableValue;
import javafx.embed.swing.SwingFXUtils;
import javafx.geometry.Side;
import javafx.scene.Node;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.MenuButton;
import javafx.scene.control.MenuItem;
import javafx.scene.control.ScrollBar;
import javafx.scene.control.Tooltip;
import javafx.scene.image.WritableImage;
import javafx.stage.FileChooser;
import javafx.beans.binding.DoubleBinding;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.CubicCurve;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import javafx.scene.text.FontPosture;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;
import javafx.stage.Stage;
import javax.imageio.ImageIO;

public class Drawer {

    private Double bound;

    private Double n;

    private Double leftHeight;

    private Double leftWidth;

    private Double wordSize;

    private Double rate1;

    private Double textToRecatngle;

    private String title;

    private double scaleFactor;

    private int width;
```

```

private int height;
private DoubleProperty buttonheight;
private DoubleProperty buttonwidth;
private Double maxWordSize;
private Double minWordSize;
private String stringFamily;
private String[] dataPaths;
private Double outHScene;
private Double outScene;
private Double minRate1;
private Double maxRate1;

public Drawer(int width, int height, Double outScene, String stringFamily, Double bound, Double n, Double
leftHeight, Double leftWidth, Double wordSize, Double rate1, Double textToRecatngle, DoubleProperty buttonheight,
DoubleProperty buttonwidth, Double maxWordSize, Double minWordSize, String[] dataPaths, Double outHScene, Double
minRate1, Double maxRate1 ){

    this.width=width;
    this.height=height;
    this.leftHeight=leftHeight;
    this.leftWidth=leftWidth;
    this.n=n;
    this.bound=bound;
    this.wordSize=wordSize;
    this.rate1=rate1;
    this.textToRecatngle=textToRecatngle;
    this.stringFamily=stringFamily;
    this.buttonheight=buttonheight;
    this.buttonwidth=buttonwidth;
    this.maxWordSize=maxWordSize;
    this.minWordSize=minWordSize;
    this.dataPaths=dataPaths;
    this.outHScene=outHScene;
    this.outScene=outScene;
    this.minRate1=minRate1;
    this.maxRate1=maxRate1;
}

public void draw(Pane pane, Pane curvePane, Pane textPane, Stage primaryStage, Scene scene){
    DoubleBinding buttonXProperty = pane.widthProperty().multiply(outScene / width);
    // cunstruct sliders
    ScrollBar neoSlider = createScrollBar("Rectangle Opacity", buttonXProperty,
pane.heightProperty().multiply((height - outHScene * 2) / height).subtract(buttonheight), -1, 1, 0);
    addReoSliderProperty(neoSlider, curvePane);
    ScrollBar textSlider = createScrollBar("Text Size", buttonXProperty,
pane.heightProperty().multiply((height-outHScene*3)/height).subtract(buttonheight), 0, 1, 0.5);

```

```

        addTextSliderProperty(textSlider ,textPane);

        ScrollBar colorSlider = createScrollBar("Curve Opacity", buttonXProperty,
pane.heightProperty().multiply((height-outHScene*4)/height).subtract(buttonheight), -1, 1, 0);

        addColorSliderProperty(colorSlider, curvePane);

        ScrollBar slopeSlider =createScrollBar("Curve Slope", buttonXProperty,
pane.heightProperty().multiply((height-outHScene*5)/height).subtract(buttonheight), 0, 1, 0.5);

        addSlopeSliderProperty(slopeSlider, curvePane);


        // construct controlBtn

        Button controlBtn = createButton("AdjustImage",buttonXProperty, pane.heightProperty().multiply((height
- outHScene) / height).subtract(buttonheight));

        addControlButtonProperty( controlBtn, reoSlider, textSlider, slopeSlider, colorSlider, pane);


        // construct fileMenuBtn

        MenuButton fileMenuButton = createFileMenuButton("SelectFile", pane, textPane,curvePane, primaryStage);

        addFileMenuButtonProperty(fileMenuButton,reoSlider,textSlider,slopeSlider, colorSlider, pane,
textPane, curvePane, primaryStage);


        // construct saveBtn

        Button saveButton =
createButton("SaveImage",pane.widthProperty().multiply(2*outScene/width).add(buttonwidth).add(buttonwidth),
pane.heightProperty().multiply((height-outHScene)/height).subtract(buttonheight));

        addSaveButtonProperty( saveButton, controlBtn, fileMenuButton,reoSlider, textSlider, slopeSlider,
colorSlider, pane, primaryStage);

        pane.getChildren().addAll(controlBtn,fileMenuButton,saveButton);

        addSceneListener(scene,textPane);
    }

    private ScrollBar createScrollBar(String tooltip, DoubleBinding layoutX, DoubleBinding layoutY, double min,
double max, double value) {

        ScrollBar scrollBar = new ScrollBar();

        scrollBar.setTooltip(new Tooltip(tooltip));

        scrollBar.setMin(min);

        scrollBar.setMax(max);

        scrollBar.setValue(value);

        scrollBar.layoutXProperty().bind(layoutX);

        scrollBar.layoutYProperty().bind(layoutY);

        return scrollBar;
    }

    private void addReoSliderProperty(ScrollBar reoSlider,Pane curvePane){

        reoSlider.valueProperty().addListener((ObservableValue<? extends Number> ov, Number old_val, Number
new_val) -> {

            double opacity = (new_val.doubleValue() + 1) / 2;

            for (Node node : curvePane.getChildren()) {

                if (node instanceof Rectangle) {

```

```

        ((Rectangle) node).setOpacity(opacity);
    }
}

});

}

private void addTextSliderProperty(ScrollBar textSlider , Pane textPane){
    textSlider.valueProperty().addListener((ObservableValue<? extends Number> ov, Number old_val, Number
new_val) -> {
        Double wordSize = (maxWordSize - minWordSize) * new_val.doubleValue() + minWordSize;
        for (Node node : textPane.getChildren()) {
            if (node instanceof Text) {
                double curX = ((Text) node).getLayoutX();
                double textWidth1 = ((Text) node).getBoundsInLocal().getWidth();
                ((Text) node).setFont(Font.font(stringFamily, FontWeight.BOLD, FontPosture.ITALIC,
wordSize));
                double textWidth2 = ((Text) node).getBoundsInLocal().getWidth();
                ((Text) node).setLayoutX(curX-textWidth2+textWidth1);
            }
        }
    });
}

private void addColorSliderProperty(ScrollBar colorSlider, Pane curvePane){
    colorSlider.valueProperty().addListener((ObservableValue<? extends Number> ov, Number old_val, Number
new_val) -> {
        double opacity = (new_val.doubleValue() + 1) / 2;
        for (Node node : curvePane.getChildren()) {
            if (node instanceof CubicCurve) {
                ((CubicCurve) node).setOpacity(opacity);
            }
        }
    });
}

private void addSlopeSliderProperty(ScrollBar slopeSlider,Pane curvePane){
    slopeSlider.valueProperty().addListener((ObservableValue<? extends Number> ov, Number old_val, Number
new_val) -> {
        Double rate1 = (maxRate1 - minRate1) * new_val.doubleValue() + minRate1;
        for (Node node : curvePane.getChildren()) {
            if (node instanceof CubicCurve) {
                CubicCurve curve = (CubicCurve) node;
                curve.controlX1Property().unbind();
                curve.controlX2Property().unbind();
                curve.controlX1Property().bind(curve.startXProperty().multiply(1-rate1).add(curve.endXPr
operty().multiply(rate1)));

```

```

        curve.controlX2Property().bind(curve.endXProperty().multiply(1-rate1).add(curve.startXProperty().multiply(rate1)));
    }
}

});

}

private Button createButton(String text, DoubleBinding layoutX, DoubleBinding layoutY) {
    Button button = new Button(text);
    button.prefWidthProperty().bind(buttonwidth);
    button.prefHeightProperty().bind(buttonheight);
    button.layoutXProperty().bind(layoutX);
    button.layoutYProperty().bind(layoutY);
    return button;
}

private void addControlButtonProperty(Button controlBtn, ScrollBar reoSlider, ScrollBar textSlider, ScrollBar slopeSlider, ScrollBar colorSlider, Pane pane){
    controlBtn.setOnAction(event -> {
        List<Node> sliders = Arrays.asList(reoSlider, textSlider, colorSlider, slopeSlider);
        if (!pane.getChildren().containsAll(sliders)) {
            pane.getChildren().addAll(sliders);
        } else {
            pane.getChildren().removeAll(sliders);
        }
    });
}

private MenuButton createFileMenuButton(String text, Pane pane, Pane textPane, Pane curvePane, Stage primaryStage) {
    MenuButton fileMenuButton = new MenuButton(text);
    fileMenuButton.prefWidthProperty().bind(buttonwidth);
    fileMenuButton.prefHeightProperty().bind(buttonheight);
    fileMenuButton.setPopupSide(Side.TOP);
    fileMenuButton.layoutXProperty().bind(pane.widthProperty().multiply(1.5 * outScene / width).add(buttonwidth));
    fileMenuButton.layoutYProperty().bind(pane.heightProperty().multiply((height - outhScene) / height).subtract(buttonheight));
    return fileMenuButton;
}

private void addFileMenuButtonProperty(MenuButton fileMenuButton, ScrollBar reoSlider, ScrollBar textSlider, ScrollBar slopeSlider, ScrollBar colorSlider, Pane pane, Pane textPane, Pane curvePane, Stage primaryStage){
    List<MenuItem> menuItems = new ArrayList<>();
    for (int i = 0; i < dataPaths.length; i++) {
        final int index = i;
        MenuItem menuItem = new MenuItem("File " + (i + 1));
    }
}

```

```

menuItem.setOnAction(event -> {
    if (pane.getChildren().containsAll(Arrays.asList(reoSlider, textSlider, colorSlider,
slopeSlider))) {
        pane.getChildren().removeAll(reoSlider, textSlider, colorSlider, slopeSlider);
    }
    pane.getChildren().removeAll(textPane, curvePane);
    textPane.getChildren().clear();
    curvePane.getChildren().clear();
    FileReader dataFile = new FileReader(dataPaths[index]);
    title = dataFile.getTitle();
    FileSorter data = new FileSorter(dataFile.getMap());
    chartDraw( primaryStage, pane, textPane, curvePane, data);
    primaryStage.setTitle(title);
});
menuItems.add(menuItem);
}
fileMenuButton.getItems().addAll(menuItems);
}

private void chartDraw(Stage primaryStage,Pane pane,Pane textPane,Pane curvePane,FileSorter data){
    Double initialleft=0.5*(height-leftHeight);
    Double startRectangle = n*outScene;
    Rectangle r1
= addRectangle(pane.widthProperty().multiply(startRectangle/width),pane.heightProperty().multiply(initiall
eft/height),pane.widthProperty().multiply(leftWidth/width),pane.heightProperty().multiply(leftHeight/height)
);
    curvePane.getChildren().add(r1);
    Text text1 =addText(data.getSortedString().get(0) + ": " +
data.getSortedInteger().get(0),pane.widthProperty().multiply((startRectangle-textToRecatngle)/width),pane.h
eightProperty().multiply((initialleft+leftHeight*0.5)/height));
    textPane.getChildren().add(text1);
    int len =data.getSortedInteger().size();
    Double right_height_location = 0.5*(height-leftHeight-(len-2)*bound);
    Double initalline=initialleft;
    for (int i=1;i<len;i++){
        Double rate = 1.0*data.getSortedInteger().get(i)/data.getSortedInteger().get(0);
        Double Rate_leftright= rate* leftHeight;
        Rectangle r2
= addRectangle(pane.widthProperty().multiply((width-leftWidth-outScene)/width),pane.heightProperty().multi
ply((right_height_location)/height),pane.widthProperty().multiply(leftWidth/width),pane.heightProperty().mu
ltiply(Rate_leftright/height));
        curvePane.getChildren().add(r2);
    }
}

```

```

        Text text2 = addText(data.getSortedString().get(i)+":
"+data.getSortedInteger().get(i),pane.widthProperty().multiply((width-outScene-leftWidth-textToRecatngle)/width),pane.heightProperty().multiply((right_height_location+Rate_lefttright*0.5)/height));

        textPane.getChildren().add(text2);

        Double startY = initalline+0.5*Rate_lefttright;

        Double endY = right_height_location+0.5*Rate_lefttright;

        CubicCurve curve1 = addCubicCurve( pane, startY, endY, startRectangle, Rate_lefttright);

        curvePane.getChildren().add(curve1);

        initalline=initalline+rate*leftHeight;

        right_height_location=right_height_location+bound+Rate_lefttright;

    }

    pane.getChildren().addAll(curvePane, textPane);

    textPane.toFront();

}

private Rectangle addRectangle(DoubleBinding xProperty,DoubleBinding yProperty,DoubleBinding
wProperty,DoubleBinding hProperty){

    Rectangle r = new Rectangle();

    r.xProperty().bind(xProperty);

    r.yProperty().bind(yProperty);

    r.widthProperty().bind(wProperty);

    r.heightProperty().bind(hProperty);

    Color color0 = new Color(Math.random(), Math.random(), Math.random(), Math.random());

    r.setStroke(color0);

    r.setFill(color0);

    return r;

}

private Text addText(String textString,DoubleBinding xProperty,DoubleBinding yProperty){

    Text text = new Text(textString);

    text.setFont(Font.font(stringFamily, FontWeight.BOLD, FontPosture.ITALIC, wordSize));

    double textWidth = text.getBoundsInLocal().getWidth();

    text.xProperty().bind(xProperty.subtract(textWidth));

    text.yProperty().bind(yProperty);

    return text;

}

private CubicCurve addCubicCurve(Pane pane,Double startY,Double endY,Double startRectangle,Double
Rate_lefttright){

    CubicCurve curve1 = new CubicCurve();

    curve1.startXProperty().bind(pane.widthProperty().multiply((startRectangle+leftWidth)/width).add(pane.heightProperty().multiply(0.5*Rate_lefttright/height)));

    curve1.startYProperty().bind(pane.heightProperty().multiply(startY/height));

    curve1.endXProperty().bind(pane.widthProperty().multiply((width-leftWidth-outScene)/width).subtract(pane.heightProperty().multiply(0.5*Rate_lefttright/height)));

    curve1.endYProperty().bind(pane.heightProperty().multiply(endY/height));

```

```

        curve1.controlX1Property().bind(curve1.startXProperty().multiply(1-rate1).add(curve1.endXProperty().
multiply(rate1)));

        curve1.controlY1Property().bind(curve1.startYProperty());

        curve1.controlX2Property().bind(curve1.endXProperty().multiply(1-rate1).add(curve1.startXProperty().
multiply(rate1)));

        curve1.controlY2Property().bind(curve1.endYProperty());

        curve1.strokeWidthProperty().bind(pane.heightProperty().multiply(Rate_leftright/height));

        Color color2 = new Color(Math.random(), Math.random(), Math.random(), Math.random());

        Color color3 = new Color(Math.random(), Math.random(), Math.random(), 0);

        curve1.setStroke(color2);

        curve1.setFill(color3);

        return curve1;

    }

    private void addSaveButtonProperty(Button saveButton, Button controlBtn, MenuButton fileMenuButton, ScrollBar
reoSlider, ScrollBar textSlider, ScrollBar slopeSlider, ScrollBar colorSlider, Pane pane, Stage primaryStage){

        saveButton.setOnAction(event -> {

            saveButton.setVisible(false);

            controlBtn.setVisible(false);

            reoSlider.setVisible(false);

            textSlider.setVisible(false);

            colorSlider.setVisible(false);

            slopeSlider.setVisible(false);

            fileMenuButton.setVisible(false);

            WritableImage image = pane.snapshot(null, null);

            FileChooser fileChooser = new FileChooser();

            fileChooser.setTitle("Save Chart");

            FileChooser.ExtensionFilter extFilterPNG = new FileChooser.ExtensionFilter("PNG files (*.png)",
"*.png");

            fileChooser.getExtensionFilters().addAll(extFilterPNG);

            File file = fileChooser.showSaveDialog(primaryStage);

            if (file != null) {

                String extension = "";

                String fileName = file.getName();

                int i = fileName.lastIndexOf('.');

                if (i > 0 && i < fileName.length() - 1) {

                    extension = fileName.substring(i + 1).toLowerCase();

                }

                try {

                    ImageIO.write(SwingFXUtils.fromFXImage(image, null), extension, file);

                } catch (IOException e) {

                    e.printStackTrace();

                }

            }

        })
    }

```

```

        saveButton.setVisible(true);
        controlBtn.setVisible(true);
        fileMenuButton.setVisible(true);
        neoSlider.setVisible(true);
        textSlider.setVisible(true);
        colorSlider.setVisible(true);
        slopeSlider.setVisible(true);
    });
}

private void addSceneListener(Scene scene, Pane textPane){
    scene.widthProperty().addListener((obs, oldVal, newVal) -> {
        scaleFactor = Math.min(newVal.doubleValue()/width, scene.getHeight()/height);
        repairText(textPane );
    });
    scene.heightProperty().addListener((obs, oldVal, newVal) -> {
        scaleFactor = Math.min(scene.getWidth()/width, newVal.doubleValue()/height);
        repairText(textPane);
    });
}

private void repairText(Pane textPane ) {
    Double wordSize = (maxWordSize+minWordSize)/2 * scaleFactor;
    for (Node node : textPane.getChildren()) {
        if (node instanceof Text) {
            double curX = ((Text) node).getLayoutX();
            double textWidth1 = ((Text) node).getBoundsInLocal().getWidth();
            ((Text) node).setFont(Font.font(stringFamily, FontWeight.BOLD, FontPosture.ITALIC, wordSize));
            double textWidth2 = ((Text) node).getBoundsInLocal().getWidth();
            ((Text) node).setLayoutX(curX-textWidth2+textWidth1);
        }
    }
}
}

```

8.3 FileReader.java

```
private Map<String, Integer> fileRead(String datapath) {  
    Map<String, Integer> map = new HashMap<>();  
    String FullEntitiy = "";  
    int sum = 0;  
    try {  
        Scanner input = new Scanner(new File(datapath));  
        int i = 0;  
        while (input.hasNextLine()) {  
            String name = input.nextLine();  
            if (name.isEmpty()) { // avoid space  
                break;  
            }  
            if (i >= 2) {  
                List<String> list = new ArrayList<>();  
                for (String splitedName : name.split(" ")) {  
                    list.add(splitedName);  
                }  
                if (list.size() == 1 && i > 2) {  
                    throw new IllegalArgumentException("Data format exception at line " + (i + 1));  
                }  
                if ( !list.get(list.size() - 1).matches("\\d+") || list.get(list.size() - 1).startsWith("-")  
|| list.get(list.size() - 1).equals("0")) {  
                    throw new IllegalArgumentException("Data must be positive");  
                }  
                int num = Integer.parseInt(list.get(list.size() - 1));  
                String nameall = "";  
                for (int indexList = 0; indexList < list.size() - 1; indexList++) {  
                    if (list.get(indexList).matches("\\d+")){  
                        throw new IllegalArgumentException("Data overflow");  
                    }  
                    nameall = nameall + " " + list.get(indexList);  
                }  
                map.put(nameall, num);  
                sum += num;  
            }  
            if (i == 1) {  
                FullEntitiy = name;  
            }  
            if (i == 0) {  
                this.title = name;  
            }  
            i++;  
        }  
    }  
}
```

```
    }  
    input.close();  
} catch (IOException | IllegalArgumentException e) {  
    System.out.println("Error: " + e.getMessage());  
    e.printStackTrace();  
    System.exit(0);  
}  
map.put(FullEntiy, sum);  
return map;  
}
```

8.4 FileSorter.java

```
package main;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

public class FileSorter {

    private List<Integer> sortedInteger = new ArrayList<>();
    private List<String> sortedString = new ArrayList<>();

    public FileSorter(Map<String, Integer> map) {

        fileSort(map);

    }

    public List<Integer> getSortedInteger() {

        return this.sortedInteger;

    }

    public List<String> getSortedString() {

        return this.sortedString;

    }

    private void fileSort(Map<String, Integer> map) {

        int len = map.values().size();

        for (int j = 0; j < len; j++) {

            int maxnum = 0;

            for (int i : map.values()) {

                if (maxnum < i) {

                    maxnum = i;

                }

            }

            sortedInteger.add(maxnum);

            for (String k : map.keySet()) {

                if (map.get(k) == maxnum) {

                    sortedString.add(k);

                    map.remove(k);

                    break;

                }

            }

        }

    }

}
```

8.5 Settings.java

```
package main;

import javafx.beans.property.DoubleProperty;
import javafx.beans.property.SimpleDoubleProperty;

public class Settings {

    private static final int WIDTH = 800;
    private static final int HEIGHT = 600;
    private static DoubleProperty buttonheight = new SimpleDoubleProperty(20);
    private static DoubleProperty buttonwidth = new SimpleDoubleProperty(91);
    private static Double wordSize = 16.0;
    private static String stringFamily = "Courier";
    private static Double rate1 = 0.5;
    private static Double bound = 0.08 * HEIGHT;
    private static String[] dataPaths = {"Example/example1.txt", "Example/example2.txt", "Example/example3.txt",
"Example/example4.txt"};

    private static Double leftwidth = 0.1 * WIDTH;
    private static Double leftheight = 0.4 * HEIGHT;
    private static Double outScene = 0.05 * WIDTH;
    private static Double outhScene = 0.05 * HEIGHT;
    private static Double n = 5.0;
    private static Double textToRectangle = 0.03 * WIDTH;
    private static final Double MIN_RATE1 = 0.1;
    private static final Double MAX_RATE1 = 0.9;
    private static final Double MIN_WORD_SIZE = 14.0;
    private static final Double MAX_WORD_SIZE = 20.0;

    public static int getWidth() {
        return WIDTH;
    }

    public static int getHeight() {
        return HEIGHT;
    }

    public static DoubleProperty getButtonHeight() {
        return buttonheight;
    }

    public static DoubleProperty getButtonWidth() {
        return buttonwidth;
    }

    public static Double getWordSize() {
        return wordSize;
    }

    public static String getStringFamily() {
        return stringFamily;
    }
}
```

```
}

public static Double getRate1() {
    return rate1;
}

public static Double getBound() {
    return bound;
}

public static String[] getDataPaths() {
    return dataPaths;
}

public static Double getLeftWidth() {
    return leftwidth;
}

public static Double getLeftHeight() {
    return leftheight;
}

public static Double getOutScene() {
    return outScene;
}

public static Double getOutHScene() {
    return outHScene;
}

public static Double getN() {
    return n;
}

public static Double getTextToRectangle() {
    return textToRectangle;
}

public static Double getMinRate1() {
    return MIN_RATE1;
}

public static Double getMaxRate1() {
    return MAX_RATE1;
}

public static Double getMinWordSize() {
    return MIN_WORD_SIZE;
}

public static Double getMaxWordSize() {
    return MAX_WORD_SIZE;
}
}
```